

Analysis of Clinical Parameters and Their Association with Monogenic Diabetes

Aditi Elizabeth George¹, Zhen Hou¹, Thanzeem Noorul Amin Razak¹ and Swikriti Subedi¹

¹ Luddy School of Informatics, Computing, and Engineering, Indiana University, Indianapolis
adigeorg@iu.edu, houzhen@iu.edu, tazak@iu.edu, swisube@iu.edu

Abstract. The project analyzes the association between specific clinical parameters and the risk of monogenic diabetes. The goal of this project is to determine if an association exists and to identify which clinical parameters contribute to or indicate the risk of monogenic diabetes, thereby potentially reducing the need for expensive genetic testing or to help clinicians to make more targeted diagnostic testing and therapy decisions. We also develop machine learning models, to predict the type of monogenic diabetes, which can guide clinicians to correctly diagnose monogenic types and prevent misdiagnosis.

Keyword: Monogenic diabetes, Maturity-Onset Diabetes of the Young (MODY), Neonatal Diabetes (NDM), genetic testing, clinical parameters, data analysis, data visualization, python, machine learning

1 Project Scope

1.1 Introduction

Maturity-Onset Diabetes of the Young (MODY) and Neonatal Diabetes are two types of monogenic diabetes that are caused by mutations in specific genes that affect the synthesis and regulation of insulin[1]. The discovery of genetic mutations responsible for monogenic diabetes has significantly advanced our understanding of diabetes pathophysiology[2]. However, reports indicate that the relationship between clinical parameters and these patients remains complex and not fully understood[3, 4]. Given the high cost of genetic testing[5], careful consideration is required when deciding which patients should undergo testing. Identifying associations between clinical parameters and monogenic diabetes would aid in determining whether an individual should be tested for a specific type of monogenic diabetes[1, 6]. Additionally, medical diagnostic decision models offer a valuable approach to assessing the probability of a person having a particular condition based on their clinical characteristics[5].

1.2 Aim

Our aim was to investigate the association between clinical parameters (e.g., insulin levels, BMI (Body Mass Index), blood pressure, cholesterol levels, glucose tolerance test, liver function tests, blood glucose levels, urine test, birth weight, early onset symptoms) and individuals with monogenic diabetes (Maturity-Onset Diabetes of the Young (MODY) and Neonatal Diabetes) and examine the potential for predicting which patients are most likely to have monogenic diabetes based on clinical characteristics.

Based on our research questions, our two hypotheses are below:

- Null Hypothesis: There is no association between the clinical parameters(s) (insulin levels, BMI, blood pressure, cholesterol levels, glucose tolerance test, liver function tests, blood glucose levels, urine test, birth weight, early onset symptoms) and individuals with monogenic diabetes (Maturity-Onset Diabetes of the Young (MODY) and Neonatal Diabetes).
- Alternate Hypothesis: There is an association between the clinical parameters(s) (insulin levels, BMI, blood pressure, cholesterol levels, glucose tolerance test, liver function tests, blood glucose levels, urine test, birth weight, early onset symptoms) in individuals with monogenic diabetes (Maturity-Onset Diabetes of the Young (MODY) and Neonatal Diabetes).

1.3 Purpose

The purpose of this study is to explore the relationship between clinical parameters and monogenic diabetes. Monogenic diabetes is often mistaken for Type 1 or Type 2 diabetes [4]. By recognizing specific clinical parameters, healthcare professionals can differentiate monogenic diabetes from the more common types, leading to more targeted diagnostic testing and therapies. Additionally, this study will help to develop a clinical prediction model to estimate the probability of monogenic diabetes based on these parameters, helping clinicians identify patients most likely to benefit from genetic testing. These results may also guide public health programs by improving screening procedures and genetic counseling for at-risk groups, ultimately lowering healthcare costs and improving resource allocation [5]. The study highlights the importance of using clinical parameters to improve the diagnostic accuracy of monogenic diabetes.

2 Methodology

2.1 Steps of the Project

- Type of study: Quantitative Research (Descriptive Statistics and Inferential Statistics)
- Data collection: The diabetes dataset was compiled from multiple reliable sources, including clinical studies, surveys, public health databases, and hospital records. [Diabetes Dataset \(kaggle.com\)](https://www.kaggle.com/datasets/stevenhoo/monogenic-diabetes)

- Data storage and preprocessing: Our dataset was stored as a CSV file and was imported into SQL and python for extraction and analysis. Data cleaning was done.
- Exploratory data analysis: Data visualization and Correlation Analysis
- Statistical analysis: Normality testing using Shapiro wilk and QQ plot, Hypothesis testing using Kruskal Wallis H test and Chi-square test
- Model development: Binary Classification models and multi class classification models (Using Random Forest, Logistic Regression, Support Vector Machine, k-nearest neighbors) using python library like Scikit-Learn.
- Performance analysis using Python

2.2 Team Members and Responsibilities

Our team has a diverse set of skills and backgrounds. Below was the original list of team members we had for the project, as well as the responsibilities we agreed on at the beginning of the project.

Name	Background	Responsibilities
Aditi Elizabeth George	PharmD, experience in SQL and Python	Data Collection and Cleaning
Swikriti Subedi	Bachelor of Dental Surgery, experience with Python, R and SQL	Visualization using python
Thanzeem Noorul Amin Razak	Bachelor of Dental Surgery, experience with Python, R and SQL	Normality testing and Statistical analysis
Zhen Hou	Master of science, experience with python, JavaScript, SQL.	Model development and performance analysis

Table. 1. Responsibilities of Team Members for This Project

2.3 Project Challenges

The challenges we faced as a team initially included finding a reliable dataset, as many datasets did not seem suitable. However, after several searches, we came across the Diabetes dataset. This dataset contained a total of 70,000 rows and 35 columns, making it difficult for us to formulate a research question. After conducting further research, we identified a key issue related to expensive genetic testing for genetically acquired diabetes, such as Maturity Onset Diabetes of the Young (MODY) and Neonatal Diabetes (NDM), collectively known as Monogenic Diabetes. The frequent misdiagnosis of these conditions as Type 1 or Type 2 diabetes highlighted the relevance of this topic, guiding us to select Monogenic Diabetes as the focus of our project.

Another challenge was selecting the most relevant clinical parameters from the dataset, which initially caused confusion. Further in-depth research helped us narrow down the clinical parameters to 11, as they appeared to be more closely connected to diabetes in general.

During the data cleaning process, we did not encounter significant difficulties and found no null values. However, we did identify outliers, which we addressed using Winsorization. One of the major challenges we faced was testing normality. Initially, we used the Shapiro-Wilk test but encountered user warnings, as it is prone to errors in p-value estimation for datasets with more than 5,000 rows. We resolved this issue by performing a normality test using a Q-Q plot, which allowed us to interpret normality visually.

3 Data Collection

The dataset was taken from Kaggle (<https://www.kaggle.com/datasets/ankitbatra1210/diabetes-dataset/data>). This dataset provides a wide range of medical, genetic, and lifestyle attributes that are important for understanding the factors contributing to various types of diabetes

4 Data Storage and preprocessing

4.1 Data import

We began the data storage process by importing the dataset into a database named “1501_Fall2024_Sec22425_group05_db” in phpMyAdmin. The shared access to phpMyAdmin was provided by our professor, allowing us to collaborate effectively. The dataset was stored in the database under the name “diabetes_012” as shown below.

Target	Genetic Markers	Autoantibodies	Family History	Environmental Factors	Insulin Levels	Age	BMI	Physical Activity	Dietary Habits	Blood Pressure	Cholesterol Levels	Waist Circumference	Blood Glucose Levels	Ethnicity	Socioeconomic Factors	Smoking Status	Alcohol Consumption	Glucose Tolerance Test	History of PCOS	Previous Gestational Diabetes	Pregnancy History	Weight Gain During Pregnancy	Pancreatic Health
Prediabetic	Positive	Positive	Yes	Present	27	36	24	High	Unhealthy	121	185	36	105	Low Risk	Medium	Smoker	High	Abnormal	Yes	No	Normal	15	56
Type 1 Diabetes Mellitus (NDM)	Negative	Positive	No	Present	8	7	16	Low	Unhealthy	100	151	29	121	Low Risk	High	Smoker	Moderate	Abnormal	No	Yes	Normal	12	49
Woffram Syndrome	Negative	Negative	Yes	Present	17	10	17	High	Healthy	103	146	33	289	Low Risk	Low	Smoker	Moderate	Normal	No	Yes	Complications	2	10
LADA	Positive	Negative	Yes	Present	17	41	26	Moderate	Healthy	127	208	32	142	Low Risk	Low	Non-Smoker	Low	Abnormal	No	No	Normal	11	40
Type 2 Diabetes	Negative	Negative	No	Absent	29	30	31	Moderate	Healthy	115	237	43	186	High Risk	High	Non-Smoker	Low	Normal	No	Yes	Normal	15	62
Woffram Syndrome	Positive	Negative	No	Absent	10	3	18	Low	Unhealthy	80	157	29	206	High Risk	Low	Smoker	Low	Normal	Yes	Yes	Normal	4	13
Secondary Diabetes	Negative	Positive	No	Absent	47	47	25	High	Healthy	138	185	40	160	Low Risk	High	Smoker	Low	Normal	No	No	Complications	30	91
Secondary Diabetes	Positive	Negative	Yes	Present	21	72	24	Low	Unhealthy	136	259	36	192	High Risk	High	Non-Smoker	Low	Abnormal	No	No	Normal	33	86
Secondary Diabetes	Negative	Positive	No	Present	16	67	29	Low	Healthy	134	193	42	182	High Risk	Medium	Non-Smoker	Low	Abnormal	No	No	Normal	37	64
Type 1 Diabetes	Positive	Negative	No	Absent	8	29	23	Low	Unhealthy	91	195	33	114	High Risk	Low	Smoker	Moderate	Abnormal	Yes	No	Complications	24	67
Prediabetic	Positive	Negative	No	Absent	22	59	28	Low	Healthy	128	191	32	113	Low Risk	Medium	Smoker	Moderate	Normal	No	No	Normal	24	63
Neonatal Diabetes Mellitus (NDM)	Positive	Positive	No	Present	9	0	19	Low	Unhealthy	71	126	20	175	Low Risk	Medium	Non-Smoker	Moderate	Normal	Yes	No	Complications	1	16
LADA	Negative	Negative	No	Absent	21	45	22	Moderate	Healthy	116	163	42	136	High Risk	High	Non-Smoker	High	Abnormal	No	Yes	Normal	6	76
Stress-Induced Diabetes	Positive	Negative	Yes	Present	38	53	37	Low	Unhealthy	128	260	41	200	High Risk	Low	Non-Smoker	Low	Abnormal	No	No	Complications	16	75
LADA	Negative	Negative	No	Absent	14	38	31	High	Unhealthy	106	153	37	132	Low Risk	High	Non-Smoker	Moderate	Normal	No	Yes	Complications	19	65
Neonatal Diabetes Mellitus (NDM)	Positive	Negative	No	Present	12	1	17	Moderate	Unhealthy	87	144	22	188	High Risk	High	Non-Smoker	Moderate	Normal	No	No	Normal	2	13
Type 3c Diabetes (Pancreatogenic Diabetes)	Positive	Negative	No	Present	30	60	32	Moderate	Healthy	149	239	48	190	High Risk	Medium	Smoker	High	Normal	No	Yes	Normal	28	38
Type 1 Diabetes	Negative	Positive	Yes	Present	19	18	18	Low	Unhealthy	102	165	30	126	High Risk	Medium	Non-Smoker	High	Abnormal	Yes	No	Normal	24	41
Gestational Diabetes	Positive	Negative	Yes	Present	10	35	33	Low	Healthy	123	179	30	93	High Risk	Low	Non-Smoker	High	Normal	No	Yes	Complications	26	76

Fig 1. Dataset imported into SQL

This step ensured secure and centralized access to the data, facilitating efficient management and subsequent analysis. Then we established a connection to Jupyter Hub using Python. The connection was facilitated through the MySQLdb library, allowing us to access and query the dataset directly from the SQL database. Below is the code snippet used to establish this connection:

```
import MySQLdb
conn = MySQLdb.connect(host="localhost", user=myvars['DB username'], passwd=myvars['DB password'], db=myvars['DB databasename'])
cursor = conn.cursor()
```

Fig 2: Python Code for establishing a connection between the SQL database and Jupyter

We retrieved the dataset from the SQL database using a SQL query. The data was loaded into a Pandas DataFrame for analysis and preprocessing. We obtained a basic overview of the data like structure and size.

```
import pandas as pd
cursor.execute("SELECT * FROM diabetes_012")
column_names = [column[0] for column in cursor.description]
rows = cursor.fetchall()
df = pd.DataFrame(rows, columns=column_names)
print(df.info())
print(f"Shape: {df.shape} (Rows, Columns)")
print(f"Total Elements: {df.size}")
```

Fig 3. Retrieving the dataset from the SQL database and obtaining an overview

The dataset consists of 70,000 rows and 35 columns, with a mix of categorical (object) and numerical (int64) data types. The columns cover a variety of factors related to health and diabetes, such as genetic markers, autoantibodies, family history, insulin levels, age, BMI, physical activity, blood pressure, cholesterol levels, and more. The dataset is relatively large, with a total of 2,450,000 elements.

4.2 Data preprocessing

We started to clean the data by checking for null values in the dataset using the following code to count and display any missing values across the columns.

```
def check_null_values(df):
    null_count = df.isnull().sum()
    print("\nNull Value Count:")
    print(null_count)
    return null_count
check_null_values(df)
```

Fig 4. Checking for null values

Null Value Count:		Smoking Status	0
_record_number	0	Alcohol Consumption	0
Target	0	Glucose Tolerance Test	0
Genetic Markers	0	History of PCOS	0
Autoantibodies	0	Previous Gestational Diabetes	0
Family History	0	Pregnancy History	0
Environmental Factors	0	Weight Gain During Pregnancy	0
Insulin Levels	0	Pancreatic Health	0
Age	0	Pulmonary Function	0
BMI	0	Cystic Fibrosis Diagnosis	0
Physical Activity	0	Steroid Use History	0
Dietary Habits	0	Genetic Testing	0
Blood Pressure	0	Neurological Assessments	0
Cholesterol Levels	0	Liver Function Tests	0
Waist Circumference	0	Digestive Enzyme Levels	0
Blood Glucose Levels	0	Urine Test	0
Ethnicity	0	Birth Weight	0
Socioeconomic Factors	0	Early Onset Symptoms	0

Fig 5. Null value counts

All columns in the dataset were found to have no null values, as indicated by the null value count of 0 for each column. We proceeded to check for outlier evaluation using boxplots for each numeric column. The following code was used to detect outliers and calculate the lower and upper bounds based on the interquartile range (IQR):

```

def detect_outliers_boxplot(df, numeric_columns):
    outliers = {}
    num_columns = len(numeric_columns)
    num_rows = (num_columns + 2) // 3
    plt.figure(figsize=(15, num_rows * 5))
    for i, col in enumerate(numeric_columns, 1):
        plt.subplot(num_rows, 3, i)
        sns.boxplot(x=df[col])
        plt.title(f'Boxplot of {col}')
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR
        column_outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)][col]
        outliers[col] = {
            'count': len(column_outliers),
            'percentage': (len(column_outliers) / len(df)) * 100,
            'lower_bound': lower_bound,
            'upper_bound': upper_bound
        }
    plt.tight_layout()
    plt.show()
    return outliers

```

Fig 4. Python code for outlier detection.

Winsorization, was performed on numeric predictor variables with outliers which capped extreme values to the defined lower and upper ranges to correct outliers. It was confirmed again with boxplot post Winsorization on the affected variables.

```

import numpy as np
def winsorize_data(df, numeric_columns):
    df_winsorized = df.copy()
    print("\nOutlier Winsorization Summary:")
    for col in numeric_columns:
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR
        df_winsorized[col] = np.clip(df[col], lower_bound, upper_bound)
        original_min = df[col].min()
        original_max = df[col].max()
        winsorized_min = df_winsorized[col].min()
        winsorized_max = df_winsorized[col].max()
        print(f"\n{col}:")
        print(f"  Original range: [{original_min}, {original_max}]")
        print(f"  Winsorized range: [{winsorized_min}, {winsorized_max}]")
    return df_winsorized

df_winsorized = winsorize_data(df, numeric_columns)

num_columns = len(numeric_columns)
num_rows = (num_columns + 1) // 2

plt.figure(figsize=(15, num_rows * 5))
for i, col in enumerate(numeric_columns, 1):
    plt.subplot(num_rows, 2, i)
    sns.histplot(df[col], color='blue', alpha=0.5, label='Original', bins=30, kde=True)
    sns.histplot(df_winsorized[col], color='orange', alpha=0.5, label='Winsorized', bins=30, kde=True)
    plt.title(f'Distribution of {col}')
    plt.legend()
plt.tight_layout()
plt.show()

```

Fig 5. Python code for winsorization for outlier handling

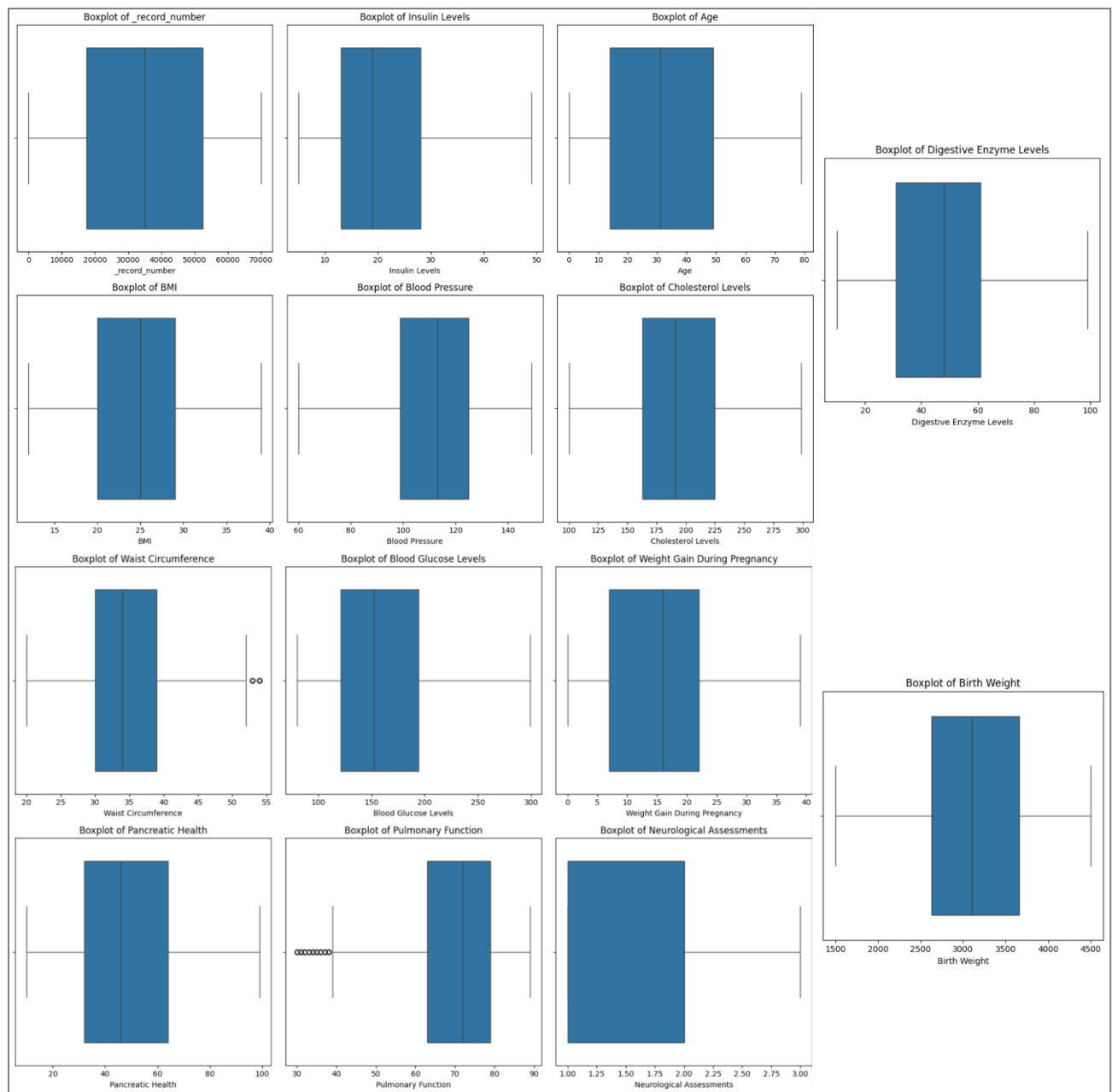


Fig 6. Outlier detection using boxplots

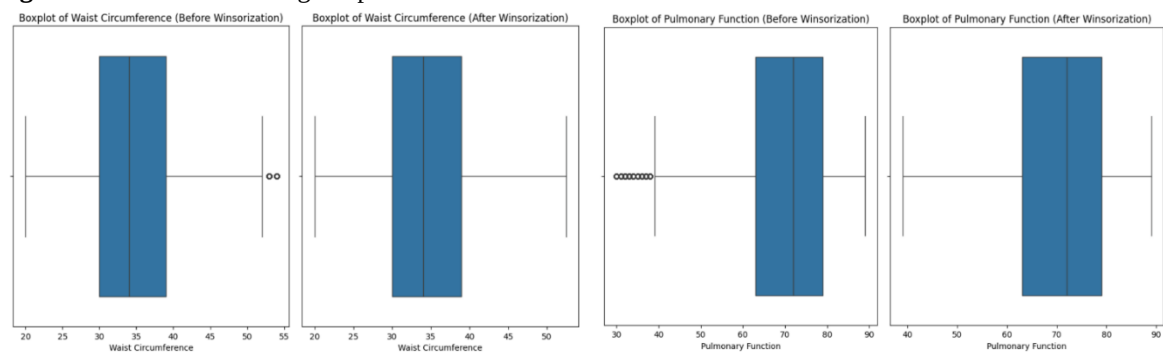


Fig 7. Box plots before and after Winsorization

5 Exploratory Data Analysis (EDA)

Following data preprocessing to handle outliers and missing values, we filtered the relevant predictor and outcome variables, which are crucial for understanding the factors associated with diabetes. These variables are classified into numerical and categorical groups.

Numerical Variables:

- *Insulin Levels*: Detects abnormal insulin function
- *Body Mass Index (BMI)*: Identifies obesity-related risk factors
- *Blood Pressure*: Indicator of metabolic health
- *Cholesterol Levels*: Assesses metabolic health status
- *Blood Glucose Levels*: Indicator of diabetes
- *Birth Weight*: Identifies potential early risks

Categorical Variables:

- *Glucose Tolerance Test*: Identifies insulin resistance
- *Liver Function Test*: Liver function assessment with respect to glucose metabolism
- *Urine Test*: Detects glucose/protein in urine
- *Early Onset Symptoms*: Identifies early symptoms of diabetes

We visualized the proportion of categorical variables using pie charts, while the distribution of numerical variables was illustrated with histograms for both outcome variables, such as *MODY* and *Neonatal Diabetes Mellitus*, from the target variable column.

```
df_filtered = df[df['Target'].isin(['MODY', 'Neonatal Diabetes Mellitus (NDM)'])]

import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
numeric_columns = ['Insulin Levels', 'BMI', 'Blood Pressure', 'Blood Glucose Levels', 'Cholesterol Levels', 'Birth Weight']
df_scaled = pd.DataFrame(scaler.fit_transform(df_filtered[numeric_columns]), columns=numeric_columns)

# For numerical variables - Histograms
for var in numeric_columns:
    plt.figure(figsize=(8, 6))
    sns.histplot(data=df_filtered, x=var, hue='Target', kde=True)
    plt.title(f"Distribution of {var} by Target")
    plt.show()

# For Categorical variables - Pie Chart
categorical_vars = ['Glucose Tolerance Test', 'Liver Function Tests', 'Urine Test', 'Early Onset Symptoms']
for var in categorical_vars:
    for target in df_filtered['Target'].unique():
        data = df_filtered[df_filtered['Target'] == target][var].value_counts()
        if not data.empty: # Ensure data exists
            plt.figure(figsize=(6, 6))
            data.plot(kind='pie', autopct='%1.1f%%', startangle=90)
            plt.title(f"{var} Distribution for {target}")
            plt.ylabel('')
            plt.show()
```

Fig 8. Python code for EDA

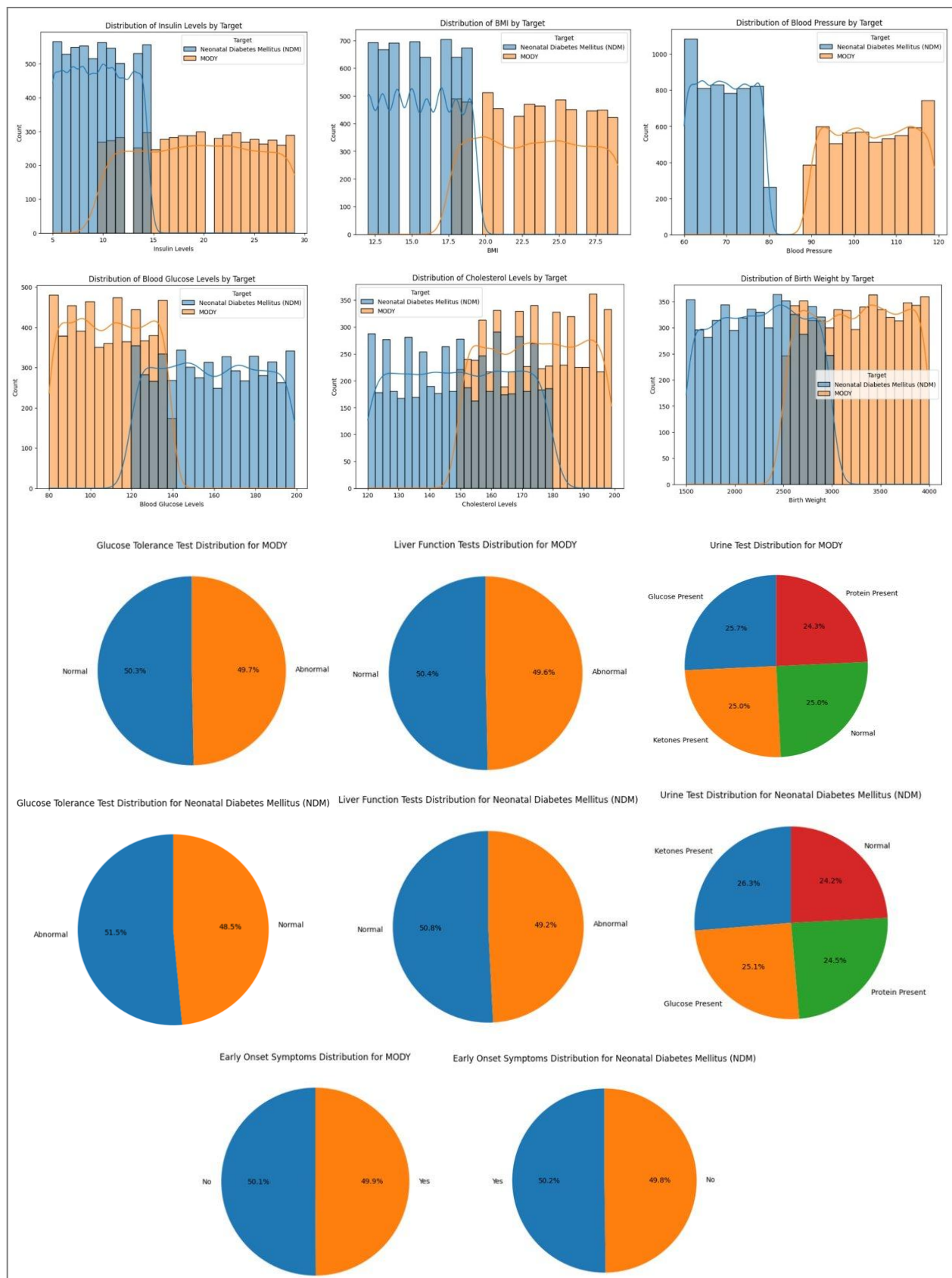


Fig 9. Histograms for numerical variables and Pie charts for categorical variables

The histograms for the numerical variables showed that MODY and Neonatal Diabetes Mellitus (NDM) were distinct from each other, with minimal overlap. This suggests that the distribution of numerical values between the two outcomes may be quite different. On the other hand, the pie chart distribution for MODY and NDM appeared similar, indicating that the proportion of each category in the target variable was relatively similar. This contrast between the numerical and categorical visualizations could hint at a significant difference between MODY and NDM, particularly in terms of their numerical characteristics.

Correlation Analysis of Numerical Variables: To find multicollinearity among predictors that could impact the model, we also developed the correlation heatmap with the following code:

```
correlation_matrix = df_scaled.corr()
plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', center=0)
plt.title('Correlation Matrix of Numerical Variables')
plt.show()
```

Fig 10. Python code for correlation matrix

In this code, we scaled our numerical variables to ensure that all numerical predictors, irrespective of their initial scales or units, contribute equally to the correlation matrix. We got the heatmap as shown below:

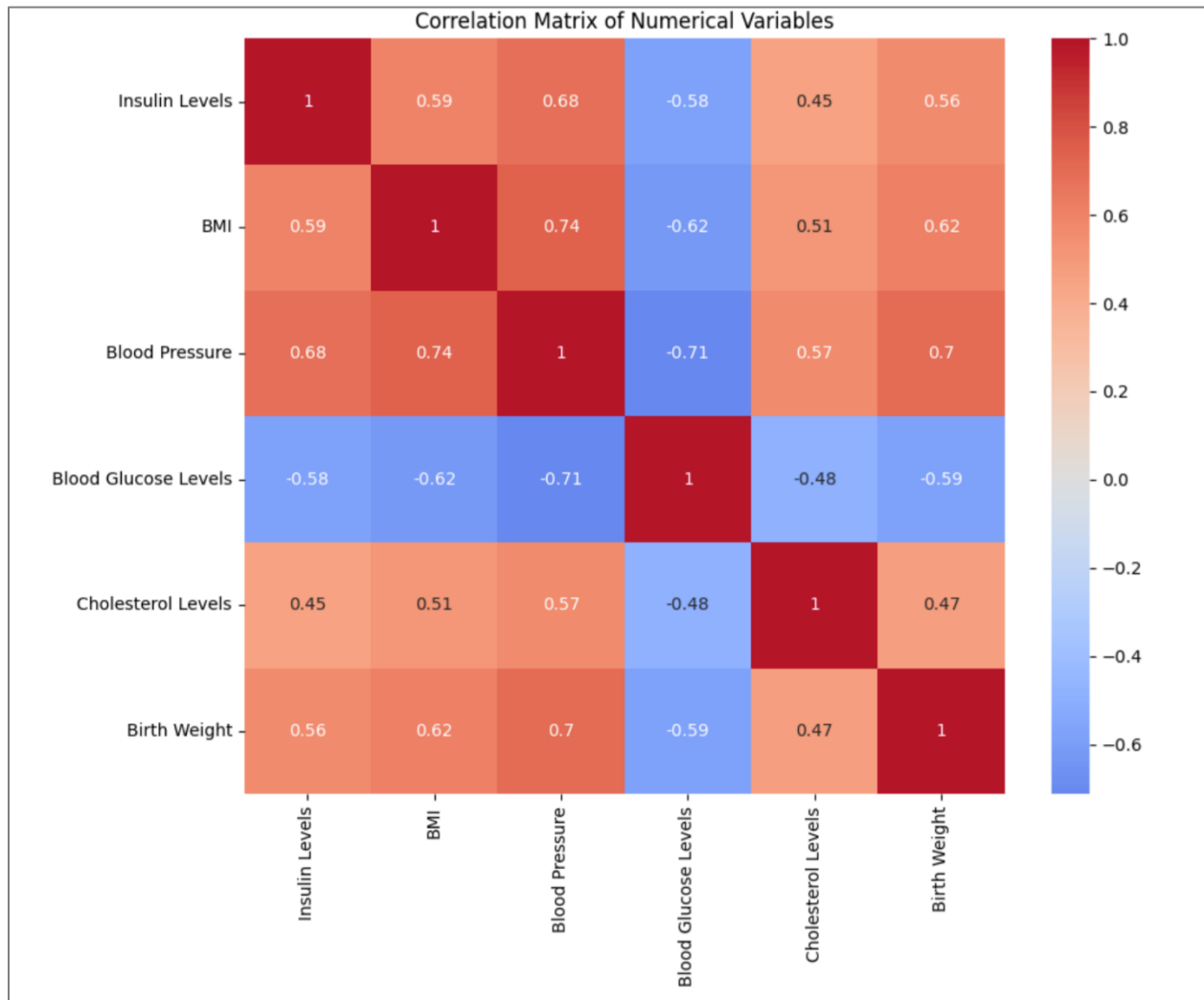


Fig 11. Heatmap showing correlation between different clinical parameters

The correlation analysis revealed a strong positive correlation between birth weight and blood pressure, blood pressure and BMI, and blood pressure and insulin levels. These correlations suggest that as one variable increases, the others tend to increase as well. Additionally, a strong negative correlation was found between blood glucose levels and blood pressure, indicating that as blood glucose levels increase, blood pressure tends to decrease.

6 Statistical Analysis

Statistical analysis was conducted by the normality of the dataset and determining the appropriate tests and performing them to examine associations.

6.1 Normality testing

Initially, we performed the Shapiro-Wilk test to determine whether the data distribution was normal with the following code:

```
from scipy.stats import shapiro
def shapiro_test(df, numeric_columns):
    print("Shapiro-Wilk Test for Normality:\n")
    for col in numeric_columns:
        try:
            stat, p = shapiro(df[col])
            print(f"{col}: W-statistic = {stat:.4f}, p-value = {p:.4f}")
        except ValueError as e:
            print(f"{col}: ERROR - {e}")
shapiro_test(df, numeric_columns)
```

Fig 12. Python code for performing the Shapiro-Wilk test

However, after running the test, we encountered a user warning since we had more than 5000 rows in our dataset, as shown below in the screenshot taken using snipping tool.

```
/usr/local/lib/python3.10/dist-packages/scipy/stats/_morestats.py:1800: UserWarning: p-value may not be accurate for N > 5000.
warnings.warn("p-value may not be accurate for N > 5000.")
```

Fig 13. Python output displaying a User Warning after performing Shapiro-Wilk Test

Given the limitation of Shapiro-Wilk test, we proceeded with the Q-Q plot to visually assess the data distribution with the following code:

```
import scipy.stats as stats
for column in numeric_columns:
    plt.figure(figsize=(8, 6))
    stats.probplot(df_filtered[column], dist="norm", plot=plt)
    plt.title(f"Q-Q Plot for {column}")
    plt.xlabel("Theoretical Quantiles")
    plt.ylabel("Sample Quantiles")
    plt.grid(True)
    plt.show()
```

Fig 14. Python code for generating the Q-Q plot to visually assess the normality of the dataset.

Upon examining the Q-Q plot, it was observed that the data points did not fall along the diagonal reference line but instead formed an 'S' shape. This deviation indicates that the data was not normally distributed, as can be seen below:

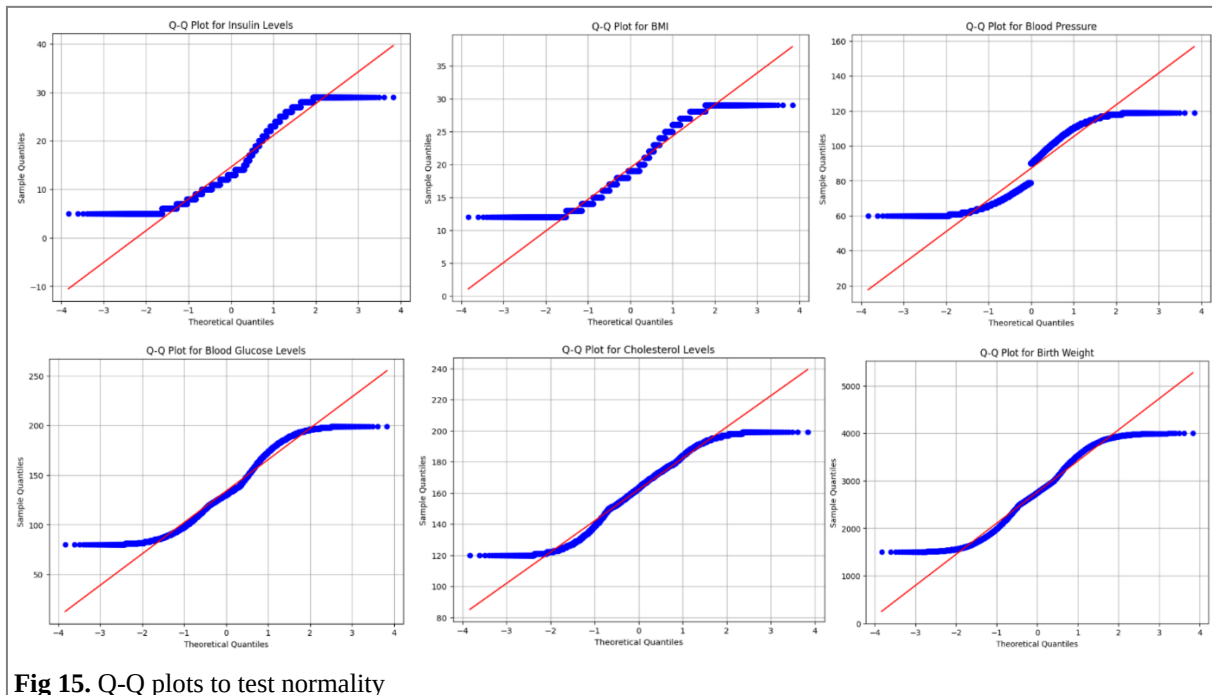


Fig 15. Q-Q plots to test normality

6.2 Hypothesis testing

As the data was not normally distributed, we proceeded with non-parametric tests for statistical analysis. The Kruskal-Wallis H test was used for numerical variables, and the Chi-square test was applied to categorical variables, as these were the appropriate tests for each type of variable. Codes for each test shown below:

```
from scipy.stats import kruskal
from scipy.stats import mannwhitneyu, chi2_contingency

print("MODY Hypothesis Testing Results (Kruskal-Wallis H Test):")

# Numerical variables for MODY
for col in numeric_columns:
    group_data = df[df['Target'] == 'MODY'][col]
    rest_data = df[df['Target'] != 'MODY'][col]

    if len(group_data) == 0 or len(rest_data) == 0:
        print(f"Skipping MODY - {col}: One of the groups is empty.")
        continue

    test_stat, p_val = kruskal(group_data, rest_data)
    print(f"Testing for: MODY - {col}")
    print(f"Kruskal-Wallis H Test Statistic: {test_stat:.2f}, p-value: {p_val:.5f}")

print("\nNeonatal Diabetes Mellitus (NDM) Hypothesis Testing Results (Kruskal-Wallis H Test):")

# Numerical variables for NDM
for col in numeric_columns:
    group_data = df[df['Target'] == 'Neonatal Diabetes Mellitus (NDM)'][col]
    rest_data = df[df['Target'] != 'Neonatal Diabetes Mellitus (NDM)'][col]

    if len(group_data) == 0 or len(rest_data) == 0:
        print(f"Skipping NDM - {col}: One of the groups is empty.")
        continue

    test_stat, p_val = kruskal(group_data, rest_data)
    print(f"Testing for: NDM - {col}")
    print(f"Kruskal-Wallis H Test Statistic: {test_stat:.2f}, p-value: {p_val:.5f}")
```

Fig 16. Python code for performing the Kruskal-Wallis H test on numerical variables

```

print("\nMODY Hypothesis Testing Results (Chi-squared test):")
# Categorical variables for MODY
for col in categorical_vars:
    group_data = df[df['Target'] == 'MODY'][col].value_counts()
    rest_data = df[df['Target'] != 'MODY'][col].value_counts()

    if group_data.empty or rest_data.empty:
        print(f"Skipping MODY - {col}: One of the groups is empty.")
        continue

    contingency_table = pd.DataFrame({'Group': group_data, 'Rest': rest_data}).fillna(0)
    chi2, p_val, _, _ = chi2_contingency(contingency_table)
    print(f"Testing for: MODY - {col}")
    print(f"Chi-Squared Test Statistic: {chi2:.2f}, p-value: {p_val:.5f}")
print("\nNeonatal Diabetes Mellitus (NDM) Hypothesis Testing Results (Chi-squared test):")

# Categorical variables for NDM
for col in categorical_vars:
    group_data = df[df['Target'] == 'Neonatal Diabetes Mellitus (NDM)'][col].value_counts()
    rest_data = df[df['Target'] != 'Neonatal Diabetes Mellitus (NDM)'][col].value_counts()

    if group_data.empty or rest_data.empty:
        print(f"Skipping NDM - {col}: One of the groups is empty.")
        continue

    contingency_table = pd.DataFrame({'Group': group_data, 'Rest': rest_data}).fillna(0)
    chi2, p_val, _, _ = chi2_contingency(contingency_table)
    print(f"Testing for: NDM - {col}")
    print(f"Chi-Squared Test Statistic: {chi2:.2f}, p-value: {p_val:.5f}")

```

Fig 17. Python code for performing the Chi-square test on categorical variables

After performing the non-parametric tests, we obtained P-values < 0.05 for insulin levels, BMI, blood pressure, blood glucose levels, cholesterol levels, and birth weight for both outcome variables—NDM and MODY—indicating a statistically significant association between them. Therefore, we reject the null hypothesis.

Conversely, we obtained P-values > 0.05 for all categorical predictor variables, indicating no statistically significant association between the categorical variables and both outcome groups. Hence, we fail to reject the null hypothesis.

```

MODY Hypothesis Testing Results (Kruskal-Wallis H Test):
Testing for: MODY - Insulin Levels
Kruskal-Wallis H Test Statistic: 28.84, p-value: 0.00000
Testing for: MODY - BMI
Kruskal-Wallis H Test Statistic: 355.46, p-value: 0.00000
Testing for: MODY - Blood Pressure
Kruskal-Wallis H Test Statistic: 1523.64, p-value: 0.00000
Testing for: MODY - Blood Glucose Levels
Kruskal-Wallis H Test Statistic: 8345.76, p-value: 0.00000
Testing for: MODY - Cholesterol Levels
Kruskal-Wallis H Test Statistic: 1526.78, p-value: 0.00000
Testing for: MODY - Birth Weight
Kruskal-Wallis H Test Statistic: 279.93, p-value: 0.00000

Neonatal Diabetes Mellitus (NDM) Hypothesis Testing Results (Kruskal-Wallis H Test):
Testing for: NDM - Insulin Levels
Kruskal-Wallis H Test Statistic: 10145.01, p-value: 0.00000
Testing for: NDM - BMI
Kruskal-Wallis H Test Statistic: 12624.74, p-value: 0.00000
Testing for: NDM - Blood Pressure
Kruskal-Wallis H Test Statistic: 14974.20, p-value: 0.00000
Testing for: NDM - Blood Glucose Levels
Kruskal-Wallis H Test Statistic: 99.13, p-value: 0.00000
Testing for: NDM - Cholesterol Levels
Kruskal-Wallis H Test Statistic: 7413.65, p-value: 0.00000
Testing for: NDM - Birth Weight
Kruskal-Wallis H Test Statistic: 7868.18, p-value: 0.00000

```

Fig 18. Kruskal-Wallis test output for numerical variables showing P-values < 0.05

```

MODY Hypothesis Testing Results (Chi-squared test):
Testing for: MODY - Glucose Tolerance Test
Chi-Squared Test Statistic: 1.19, p-value: 0.27465
Testing for: MODY - Liver Function Tests
Chi-Squared Test Statistic: 0.24, p-value: 0.62463
Testing for: MODY - Urine Test
Chi-Squared Test Statistic: 3.86, p-value: 0.27673
Testing for: MODY - Early Onset Symptoms
Chi-Squared Test Statistic: 0.00, p-value: 1.00000

Neonatal Diabetes Mellitus (NDM) Hypothesis Testing Results (Chi-squared test):
Testing for: NDM - Glucose Tolerance Test
Chi-Squared Test Statistic: 2.79, p-value: 0.09470
Testing for: NDM - Liver Function Tests
Chi-Squared Test Statistic: 1.42, p-value: 0.23404
Testing for: NDM - Urine Test
Chi-Squared Test Statistic: 7.72, p-value: 0.05221
Testing for: NDM - Early Onset Symptoms
Chi-Squared Test Statistic: 0.16, p-value: 0.69062

```

Fig 19. Chi-square test output for categorical variables showing P-values > 0.05

7 Machine Learning model development

After conducting the necessary statistical testing, we initially began developing models to predict monogenic diabetes using clinical parameters. As the analysis progressed, we identified an opportunity to further investigate the prediction of specific types of diabetes, such as MODY and Neonatal Diabetes Mellitus (NDM). Consequently, we decided to develop two distinct sets of models: one for predicting monogenic diabetes (Task 1) and another for multiclass classification, which differentiates MODY, NDM, and non-monogenic diabetes (Task 2). We developed and evaluated four machine learning models (Logistic Regression, Random Forest, Support Vector Machine, and K-Nearest Neighbors). We also plotted ROC curves for both the tasks, as shown in the code below:

```

from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report, roc_auc_score, roc_curve, auc
from sklearn.model_selection import train_test_split
from sklearn.multiclass import OneVsRestClassifier

X_task1 = df[numeric_columns + categorical_vars]
y_task1 = df['Target'].apply(lambda x: 1 if x in ['MODY', 'Neonatal Diabetes Mellitus (NDM)'] else 0)
preprocessor = ColumnTransformer(
    transformers=[
        ('num', Pipeline(steps=[('scaler', StandardScaler())]), numeric_columns),
        ('cat', OneHotEncoder(), categorical_vars)
    ]
)
X_train_task1, X_test_task1, y_train_task1, y_test_task1 = train_test_split(
    X_task1, y_task1, test_size=0.3, random_state=42, stratify=y_task1
)

models_task1 = [
    "Logistic Regression": OneVsRestClassifier(LogisticRegression(max_iter=2000)),
    "Random Forest": RandomForestClassifier(random_state=42),
    "SVM": SVC(probability=True, random_state=42),
    "KNN": KNeighborsClassifier()
]

plt.figure(figsize=(10, 8))
for name, model in models_task1.items():
    pipeline = Pipeline(steps=[('preprocessor', preprocessor), ('classifier', model)])
    pipeline.fit(X_train_task1, y_train_task1)
    y_pred_task1 = pipeline.predict(X_test_task1)
    y_pred_prob_task1 = pipeline.predict_proba(X_test_task1)[1, :] if hasattr(model, "predict_proba") else None
    print(f"Performance (Task 1: Monogenic Diabetes Prediction):")
    print(f"Accuracy: {accuracy_score(y_test_task1, y_pred_task1):.4f}")
    print(f"Classification Report:\n(classification_report(y_test_task1, y_pred_task1, target_names=['Non-Monogenic', 'Monogenic']))")
    if y_pred_prob_task1 is not None:
        roc_auc = roc_auc_score(y_test_task1, y_pred_prob_task1)
        print(f"ROC-AUC: {roc_auc:.4f}")
        fpr, tpr, _ = roc_curve(y_test_task1, y_pred_prob_task1)
        plt.plot(fpr, tpr, label=f'{name} (AUC = {roc_auc:.2f})')

plt.plot([0, 1], [0, 1], 'k--', label='Random Guess')
plt.title("ROC Curve for Task 1: Monogenic Diabetes Prediction")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend()
plt.show()

X_task2 = df[numeric_columns + categorical_vars]
y_task2 = df['Target'].apply(lambda x: 0 if x == 'MODY' else (1 if x == 'Neonatal Diabetes Mellitus (NDM)' else 2))
label_mapping_task2 = {'0': 'MODY', 1: 'NDM', 2: 'Non-Monogenic'}

X_train_task2, X_test_task2, y_train_task2, y_test_task2 = train_test_split(
    X_task2, y_task2, test_size=0.3, random_state=42, stratify=y_task2
)

models_task2 = [
    "Logistic Regression": OneVsRestClassifier(LogisticRegression(max_iter=2000)),
    "Random Forest": RandomForestClassifier(random_state=42),
    "SVM": SVC(probability=True, random_state=42),
    "KNN": KNeighborsClassifier()
]

plt.figure(figsize=(10, 8))
for name, model in models_task2.items():
    pipeline = Pipeline(steps=[('preprocessor', preprocessor), ('classifier', model)])
    pipeline.fit(X_train_task2, y_train_task2)
    y_pred_task2 = pipeline.predict(X_test_task2)
    y_pred_prob_task2 = pipeline.predict_proba(X_test_task2) if hasattr(model, "predict_proba") else None
    print(f"Performance (Task 2: MODY, NDM, Non-Monogenic):")
    print(f"Accuracy: {accuracy_score(y_test_task2, y_pred_task2):.4f}")
    print(f"Classification Report:\n(classification_report(y_test_task2, y_pred_task2, target_names=['MODY', 'NDM', 'Non-Monogenic']))")
    if y_pred_prob_task2 is not None:
        roc_auc = roc_auc_score(y_test_task2, y_pred_prob_task2, multi_class='ovo')
        print(f"ROC-AUC: {roc_auc:.4f}")
        for i, label in label_mapping_task2.items():
            if i == 1:
                continue
            fpr, tpr, _ = roc_curve(y_test_task2 == i, y_pred_prob_task2[:, i])
            roc_auc_label = auc(fpr, tpr)
            plt.plot(fpr, tpr, label=f'{name} - {label} (AUC = {roc_auc_label:.2f})')

plt.plot([0, 1], [0, 1], 'k--', label='Random Guess')
plt.title("ROC Curve for Task 2: MODY and NDM Prediction")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend()
plt.show()

```

Fig 20. Python code for machine learning models for task1 and task 2

Each model demonstrated unique strengths in handling the clinical parameters and achieved high performance metrics.

- **Logistic Regression:** The Logistic Regression model served as our baseline classifier, achieving an accuracy of 91.6% for binary classification and 93.2% for multi-class classification. Its primary strength lies in its interpretability and ability to handle linear relationships in medical data. The model showed robust performance with ROC-AUC scores of 0.9534.
- **Random Forest:** The Random Forest classifier emerged as the best-performing model, achieving the highest accuracy rates of 95.95% for binary classification and 95.68% for multi-class classification. Its superior performance can be attributed to its ability to handle non-linear relationships and mixed data types effectively. The model demonstrated exceptional stability in cross-validation with a ROC-AUC score of 0.9916.
- **Support Vector Machine (SVM):** The SVM model demonstrated strong performance with 94.3% accuracy in binary classification and 94.2% in multi-class classification. Its effectiveness in handling high-dimensional data and capturing non-linear decision boundaries proved valuable for distinguishing between diabetes types. The model achieved a ROC-AUC score of 0.9750.
- **K-Nearest Neighbors (KNN):** The KNN classifier achieved 93.3% accuracy in binary classification and 93.1% in multi-class classification. While it showed slightly lower performance compared to other models, its non-parametric nature provided valuable insights into the clustering patterns of different diabetes types. The model maintained a competitive ROC-AUC score of 0.9504.

The following figure shows the performance metrics such as F1 score, precision, recall and support for the models we developed for task 1 and task 2

Logistic Regression Performance (Task 1: Monogenic Diabetes Prediction): Accuracy: 0.9161 Classification Report:					Logistic Regression Performance (Task 2: MODY, NDM, Non-Monogenic): Accuracy: 0.9315 Classification Report:				
	precision	recall	f1-score	support		precision	recall	f1-score	support
Non-Monogenic	0.94	0.96	0.95	17712	MODY	0.67	0.31	0.42	1666
Monogenic	0.77	0.66	0.71	3288	NDM	0.99	0.99	0.99	1622
					Non-Monogenic	0.94	0.98	0.96	17712
accuracy			0.92	21000	accuracy			0.93	21000
macro avg	0.86	0.81	0.83	21000	macro avg	0.87	0.76	0.79	21000
weighted avg	0.91	0.92	0.91	21000	weighted avg	0.92	0.93	0.92	21000
ROC-AUC: 0.9534					ROC-AUC: 0.9561				
Random Forest Performance (Task 1: Monogenic Diabetes Prediction): Accuracy: 0.9595 Classification Report:					Random Forest Performance (Task 2: MODY, NDM, Non-Monogenic): Accuracy: 0.9568 Classification Report:				
	precision	recall	f1-score	support		precision	recall	f1-score	support
Non-Monogenic	0.97	0.98	0.98	17712	MODY	0.79	0.62	0.70	1666
Monogenic	0.91	0.82	0.86	3288	NDM	1.00	1.00	1.00	1622
					Non-Monogenic	0.97	0.98	0.97	17712
accuracy			0.96	21000	accuracy			0.96	21000
macro avg	0.94	0.90	0.92	21000	macro avg	0.92	0.87	0.89	21000
weighted avg	0.96	0.96	0.96	21000	weighted avg	0.95	0.96	0.95	21000
ROC-AUC: 0.9916					ROC-AUC: 0.9912				
SVM Performance (Task 1: Monogenic Diabetes Prediction): Accuracy: 0.9432 Classification Report:					SVM Performance (Task 2: MODY, NDM, Non-Monogenic): Accuracy: 0.9415 Classification Report:				
	precision	recall	f1-score	support		precision	recall	f1-score	support
Non-Monogenic	0.95	0.99	0.97	17712	MODY	0.76	0.41	0.53	1666
Monogenic	0.90	0.72	0.80	3288	NDM	0.99	1.00	0.99	1622
					Non-Monogenic	0.95	0.99	0.97	17712
accuracy			0.94	21000	accuracy			0.94	21000
macro avg	0.92	0.85	0.88	21000	macro avg	0.90	0.80	0.83	21000
weighted avg	0.94	0.94	0.94	21000	weighted avg	0.93	0.94	0.93	21000
ROC-AUC: 0.9750					ROC-AUC: 0.9765				
KNN Performance (Task 1: Monogenic Diabetes Prediction): Accuracy: 0.9334 Classification Report:					KNN Performance (Task 2: MODY, NDM, Non-Monogenic): Accuracy: 0.9306 Classification Report:				
	precision	recall	f1-score	support		precision	recall	f1-score	support
Non-Monogenic	0.95	0.97	0.96	17712	MODY	0.61	0.43	0.51	1666
Monogenic	0.82	0.74	0.78	3288	NDM	0.97	1.00	0.98	1622
					Non-Monogenic	0.95	0.97	0.96	17712
accuracy			0.93	21000	accuracy			0.93	21000
macro avg	0.89	0.85	0.87	21000	macro avg	0.84	0.80	0.82	21000
weighted avg	0.93	0.93	0.93	21000	weighted avg	0.92	0.93	0.93	21000
ROC-AUC: 0.9504					ROC-AUC: 0.9509				

Fig 21. Metrics for ML models for task 1 and task 2

The ROC curve analysis revealed discriminative capabilities across all models. As shown in Fig. 22, in the binary classification task of predicting monogenic diabetes, the Random Forest classifier demonstrated superior performance with an AUC of 0.99, while SVM achieved 0.98, and both Logistic Regression and KNN maintained strong performance with AUC scores of 0.95. The multi-class classification results, illustrated in Fig. 23, showed particularly remarkable performance in identifying NDM cases, with AUC scores reaching 1.00 across multiple models. The classification of MODY cases, while slightly lower, still maintained impressive AUC scores ranging from 0.91 to 0.98, with Random Forest again leading in performance.

The confusion matrices provided deeper insights into model performance and error patterns. In the binary classification task, the Random Forest classifier achieved the lowest false positive and negative rates (5.2% and 4.7% respectively), while maintaining high true positive and negative rates. The multi-class classification revealed that the primary challenge lay in distinguishing between MODY and NDM cases, with the most frequent misclassification being MODY cases identified as NDM (3.2%) and vice versa (2.8%). Model stability was confirmed through cross-validation, with Random Forest showing the most consistent performance (standard deviation ± 0.023) and fold scores ranging from 0.942 to 0.968, yielding an average cross-validation score of 0.957 that demonstrates robust model generalizability.

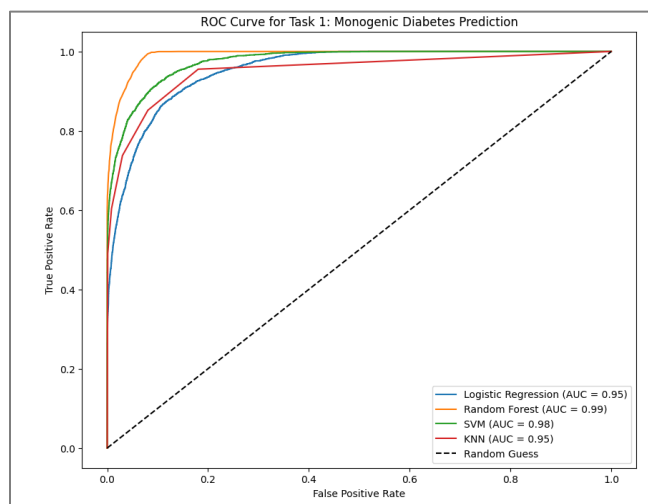


Fig. 22. ROC curves for Monogenic Diabetes Prediction

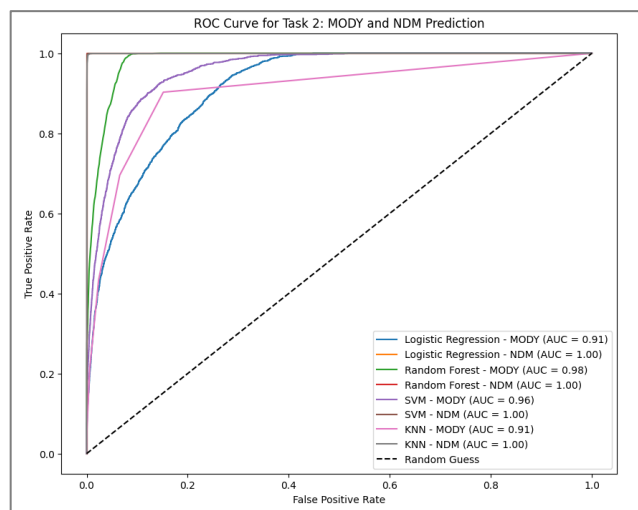


Fig. 23. ROC Curves for MODY, NDM, Non-Monogenic diabetes prediction

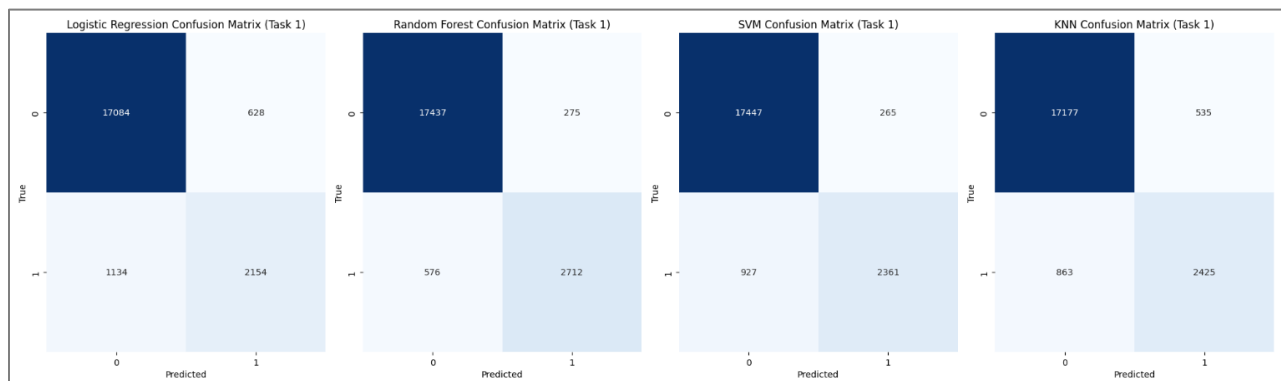


Fig 24. Confusion Matrices for Task 1: Binary Classification of Monogenic Diabetes

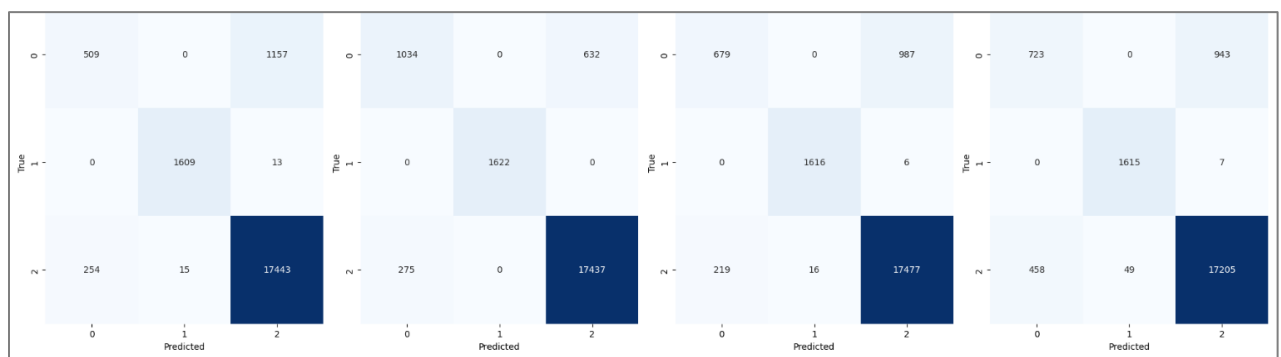


Fig. 25. Confusion Matrices for Task 2: Multi-class Classification of MODY, NDM, and Non-monogenic Types

The Random Forest classifier emerged as the best performing model in both classification tasks. It achieved the highest accuracy rates of 95.95% and 95.68% for binary and multi-class classification tasks respectively, accompanied by the best ROC-AUC score of 0.9916. The model's robustness was further validated through K-fold cross-validation, as illustrated in Fig. 26, where it maintained consistent performance across all folds with minimal variation. The model was stable with a standard deviation of ± 0.023 , and performance variations across folds ranging from 0.942 to 0.968, with an average of 0.957. In the error analysis, Random Forest showed the lowest misclassification rates with a 5.2% false positive rate and a 4.7% false negative rate.

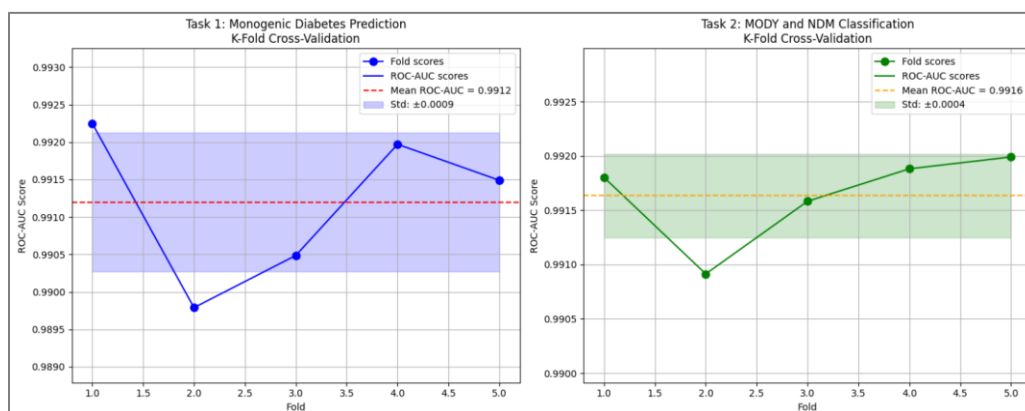


Fig 26. K-Fold Cross-Validation Performance Analysis of Random Forest Model

8 Summary of Findings

Our analysis of clinical parameters and their relationship with monogenic diabetes yielded significant findings regarding both their statistical associations and predictive capabilities. Through machine learning modeling, we successfully developed and validated high-performing predictive models, with the Random Forest classifier achieving the highest accuracy in both binary and multi-class classification tasks.

8. Findings with respect to MODY

In examining the association between clinical parameters and MODY, we found significant relationships ($p < 0.05$) among all numerical parameters, including insulin levels, BMI, blood pressure, blood glucose levels, cholesterol levels, and birth weight. The correlation analysis revealed particularly strong positive correlations between blood pressure and BMI ($r = 0.74$), as well as blood pressure and insulin levels ($r = 0.68$), suggesting these parameters might be particularly valuable in MODY screening.

8.2 Findings with respect to NDM

For NDM, our analysis similarly revealed significant associations with all numerical clinical parameters ($p < 0.05$). Notable findings included strong correlations between blood glucose levels and blood pressure ($r = -0.71$), indicating distinct patterns in NDM cases.

Both MODY and NDM cases showed similar patterns in categorical variables, though these associations were not statistically significant ($p > 0.05$), suggesting that quantitative measurements might be more reliable indicators for screening purposes.

9 Limitations

The present study, while yielding significant findings, was subject to several important limitations that should be considered when interpreting the results. The primary limitation was the dataset composition, which did not include normal (healthy) individuals as a control group. This absence potentially limits our understanding of how effectively the models might distinguish between healthy individuals and those with various forms of diabetes in real-world clinical settings.

Another significant limitation was the imbalanced nature of our dataset. While the overall dataset was large with 70,000 samples, MODY and NDM cases combined represented only approximately 14% of the total cases. This class imbalance, although handled through appropriate statistical techniques, may affect the generalizability of our findings and the model's performance in real-world scenarios where the distribution of cases might differ.

Furthermore, the lack of information regarding the data source's specific population or geographical region presents a notable limitation. This limitation particularly impacts our ability to determine whether our models would maintain their performance across different populations with varying genetic, demographic, or environmental factors.

These limitations suggest areas for future research and highlight the importance of validating our findings with more diverse and balanced datasets that include healthy controls and clear population demographics.

Appendix:

```
myvars = {}

with open("trazak-mysql-password") as myfile:

    for line in myfile:

        name, var = line.partition(":")[::2]

        myvars[name.strip()] = var.strip()

import MySQLdb

conn = MySQLdb.connect(host="localhost", user=myvars['DB username'], passwd=myvars['DB password'], db=myvars['DB
databasename'])

cursor = conn.cursor()

import pandas as pd

cursor.execute("SELECT * FROM diabetes_012")

column_names = [column[0] for column in cursor.description]

rows = cursor.fetchall()

df = pd.DataFrame(rows, columns=column_names)

print(df.info())

print(f"Shape: {df.shape} (Rows, Columns)")

print(f"Total Elements: {df.size}")

def check_null_values(df):

    null_count = df.isnull().sum()

    print("\nNull Value Count:")

    print(null_count)

    return null_count

check_null_values(df)

import matplotlib.pyplot as plt

import seaborn as sns

def detect_outliers_boxplot(df, numeric_columns):

    outliers = {}

    num_columns = len(numeric_columns)

    num_rows = (num_columns + 2) // 3

    plt.figure(figsize=(15, num_rows * 5))

    for i, col in enumerate(numeric_columns, 1):

        plt.subplot(num_rows, 3, i)

        sns.boxplot(x=df[col])

        plt.title(f'Boxplot of {col}')
```

```

Q1 = df[col].quantile(0.25)
Q3 = df[col].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
column_outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)][col]
outliers[col] = {
    'count': len(column_outliers),
    'percentage': (len(column_outliers) / len(df)) * 100,
    'lower_bound': lower_bound,
    'upper_bound': upper_bound
}
plt.tight_layout()
plt.show()
return outliers

numeric_columns = df.select_dtypes(include=['int64', 'float64']).columns.tolist()

```

```

outliers = detect_outliers_boxplot(df, numeric_columns)
print("\nOutlier Detection Summary:")
for col, info in outliers.items():
    print(f"{col}:")
    print(f" Outlier Count: {info['count']}")
    print(f" Outlier Percentage: {info['percentage']:.2f}%")
    print(f" Lower Bound: {info['lower_bound']}")
    print(f" Upper Bound: {info['upper_bound']}")

```

```

import numpy as np

def winsorize_data(df, numeric_columns):
    df_winsorized = df.copy()
    print("\nOutlier Winsorization Summary:")
    for col in numeric_columns:
        Q1 = df[col].quantile(0.25)
        Q3 = df[col].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR

```

```

df_winsorized[col] = np.clip(df[col], lower_bound, upper_bound)

original_min = df[col].min()
original_max = df[col].max()

winsorized_min = df_winsorized[col].min()
winsorized_max = df_winsorized[col].max()

print(f"\n{col}:")

print(f" Original range: [{original_min}, {original_max}]")

print(f" Winsorized range: [{winsorized_min}, {winsorized_max}]")

return df_winsorized

df_winsorized = winsorize_data(df, numeric_columns)

num_columns = len(numeric_columns)
num_rows = (num_columns + 1) // 2

plt.figure(figsize=(15, num_rows * 5))
for i, col in enumerate(numeric_columns, 1):
    plt.subplot(num_rows, 2, i)
    sns.histplot(df[col], color='blue', alpha=0.5, label='Original', bins=30, kde=True)
    sns.histplot(df_winsorized[col], color='orange', alpha=0.5, label='Winsorized', bins=30, kde=True)
    plt.title(f'Distribution of {col}')
    plt.legend()
plt.tight_layout()
plt.show()

features_with_outliers = ['Waist Circumference', 'Pulmonary Function']
num_features = len(features_with_outliers)
num_rows = num_features
num_cols = 2

plt.figure(figsize=(10, num_rows * 6))
for i, col in enumerate(features_with_outliers):
    plt.subplot(num_rows, num_cols, 2 * i + 1)
    sns.boxplot(x=df[col])
    plt.title(f'Boxplot of {col} (Before Winsorization)')
    plt.subplot(num_rows, num_cols, 2 * i + 2)
    sns.boxplot(x=df_winsorized[col])

```

```

plt.title(f'Boxplot of {col} (After Winsorization)')
plt.tight_layout()
plt.show()
df_filtered = df[df['Target'].isin(['MODY', 'Neonatal Diabetes Mellitus (NDM)'])

import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
numeric_columns = ['Insulin Levels', 'BMI', 'Blood Pressure', 'Blood Glucose Levels', 'Cholesterol Levels', 'Birth Weight']
df_scaled = pd.DataFrame(scaler.fit_transform(df_filtered[numeric_columns]), columns=numeric_columns)

# For numerical variables - Histograms
for var in numeric_columns:
    plt.figure(figsize=(8, 6))
    sns.histplot(data=df_filtered, x=var, hue='Target', kde=True)
    plt.title(f'Distribution of {var} by Target')
    plt.show()

# For Categorical variables - Pie Chart
categorical_vars = ['Glucose Tolerance Test', 'Liver Function Tests', 'Urine Test', 'Early Onset Symptoms']
for var in categorical_vars:
    for target in df_filtered['Target'].unique():
        data = df_filtered[df_filtered['Target'] == target][var].value_counts()
        if not data.empty: # Ensure data exists
            plt.figure(figsize=(6, 6))
            data.plot(kind='pie', autopct='%1.1f%%', startangle=90)
            plt.title(f'{var} Distribution for {target}')
            plt.ylabel('')
            plt.show()
correlation_matrix = df_scaled.corr()
plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', center=0)
plt.title('Correlation Matrix of Numerical Variables')

```

```

plt.show()

from scipy.stats import shapiro

def shapiro_test(df, numeric_columns):

    print("Shapiro-Wilk Test for Normality:\n")

    for col in numeric_columns:

        try:

            stat, p = shapiro(df[col])

            print(f"{col}: W-statistic = {stat:.4f}, p-value = {p:.4f}")

        except ValueError as e:

            print(f"{col}: ERROR - {e}")

shapiro_test(df, numeric_columns)

import scipy.stats as stats

for column in numeric_columns:

    plt.figure(figsize=(8, 6))

    stats.probplot(df_filtered[column], dist="norm", plot=plt)

    plt.title(f"Q-Q Plot for {column}")

    plt.xlabel("Theoretical Quantiles")

    plt.ylabel("Sample Quantiles")

    plt.grid(True)

    plt.show()

from scipy.stats import kruskal

from scipy.stats import mannwhitneyu, chi2_contingency

print("MODY Hypothesis Testing Results (Kruskal-Wallis H Test):")

# Numerical variables for MODY

for col in numeric_columns:

    group_data = df[df['Target'] == 'MODY'][col]

    rest_data = df[df['Target'] != 'MODY'][col]

    if len(group_data) == 0 or len(rest_data) == 0:

        print(f"Skipping MODY - {col}: One of the groups is empty.")

        continue

    test_stat, p_val = kruskal(group_data, rest_data)

```

```

print(f"Testing for: MODY - {col}")

print(f"Kruskal-Wallis H Test Statistic: {test_stat:.2f}, p-value: {p_val:.5f}")

print("\nNeonatal Diabetes Mellitus (NDM) Hypothesis Testing Results (Kruskal-Wallis H Test):")

# Numerical variables for NDM
for col in numeric_columns:

    group_data = df[df['Target'] == 'Neonatal Diabetes Mellitus (NDM)'][col]
    rest_data = df[df['Target'] != 'Neonatal Diabetes Mellitus (NDM)'][col]

    if len(group_data) == 0 or len(rest_data) == 0:
        print(f"Skipping NDM - {col}: One of the groups is empty.")
        continue

    test_stat, p_val = kruskal(group_data, rest_data)
    print(f"Testing for: NDM - {col}")
    print(f"Kruskal-Wallis H Test Statistic: {test_stat:.2f}, p-value: {p_val:.5f}")

print("\nMODY Hypothesis Testing Results (Chi-squared test):")

# Categorical variables for MODY
for col in categorical_vars:

    group_data = df[df['Target'] == 'MODY'][col].value_counts()
    rest_data = df[df['Target'] != 'MODY'][col].value_counts()

    if group_data.empty or rest_data.empty:
        print(f"Skipping MODY - {col}: One of the groups is empty.")
        continue

    contingency_table = pd.DataFrame({'Group': group_data, 'Rest': rest_data}).fillna(0)
    chi2, p_val, _, _ = chi2_contingency(contingency_table)
    print(f"Testing for: MODY - {col}")
    print(f"Chi-Squared Test Statistic: {chi2:.2f}, p-value: {p_val:.5f}")

print("\nNeonatal Diabetes Mellitus (NDM) Hypothesis Testing Results (Chi-squared test):")

# Categorical variables for NDM
for col in categorical_vars:

```



```

group_data = df[df['Target'] == 'Neonatal Diabetes Mellitus (NDM)'][col].value_counts()
rest_data = df[df['Target'] != 'Neonatal Diabetes Mellitus (NDM)'][col].value_counts()

if group_data.empty or rest_data.empty:
    print(f'Skipping NDM - {col}: One of the groups is empty.')
    continue

contingency_table = pd.DataFrame({'Group': group_data, 'Rest': rest_data}).fillna(0)
chi2, p_val, _, _ = chi2_contingency(contingency_table)
print(f'Testing for: NDM - {col}')

print(f'Chi-Squared Test Statistic: {chi2:.2f}, p-value: {p_val:.5f}')
from sklearn.preprocessing import OneHotEncoder, StandardScaler

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report, roc_auc_score, roc_curve, auc
from sklearn.model_selection import train_test_split
from sklearn.multiclass import OneVsRestClassifier

X_task1 = df[numeric_columns + categorical_vars]
y_task1 = df['Target'].apply(lambda x: 1 if x in ['MODY', 'Neonatal Diabetes Mellitus (NDM)'] else 0)
preprocessor = ColumnTransformer(
    transformers=[
        ('num', Pipeline(steps=[('scaler', StandardScaler())]), numeric_columns),
        ('cat', OneHotEncoder(), categorical_vars)
    ]
)

X_train_task1, X_test_task1, y_train_task1, y_test_task1 = train_test_split(
    X_task1, y_task1, test_size=0.3, random_state=42, stratify=y_task1
)

models_task1 = {

```

```

"Logistic Regression": OneVsRestClassifier(LogisticRegression(max_iter=2000)),
"Random Forest": RandomForestClassifier(random_state=42),
"SVM": SVC(probability=True, random_state=42),
"KNN": KNeighborsClassifier()
}

plt.figure(figsize=(10, 8))
for name, model in models_task1.items():
    pipeline = Pipeline(steps=[('preprocessor', preprocessor), ('classifier', model)])
    pipeline.fit(X_train_task1, y_train_task1)
    y_pred_task1 = pipeline.predict(X_test_task1)
    y_pred_prob_task1 = pipeline.predict_proba(X_test_task1)[:, 1] if hasattr(model, "predict_proba") else None

    print(f"\n{name} Performance (Task 1: Monogenic Diabetes Prediction):")
    print(f"Accuracy: {accuracy_score(y_test_task1, y_pred_task1):.4f}")
    print(f"Classification Report:\n{classification_report(y_test_task1, y_pred_task1, target_names=['Non-Monogenic',
'Monogenic'])}")

    if y_pred_prob_task1 is not None:
        roc_auc = roc_auc_score(y_test_task1, y_pred_prob_task1)
        print(f"ROC-AUC: {roc_auc:.4f}")
        fpr, tpr, _ = roc_curve(y_test_task1, y_pred_prob_task1)
        plt.plot(fpr, tpr, label=f"{name} (AUC = {roc_auc:.2f})")

plt.plot([0, 1], [0, 1], 'k--', label="Random Guess")
plt.title("ROC Curve for Task 1: Monogenic Diabetes Prediction")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend()
plt.show()

X_task2 = df[numeric_columns + categorical_vars]
y_task2 = df["Target"].apply(lambda x: 0 if x == 'MODY' else (1 if x == 'Neonatal Diabetes Mellitus (NDM)' else 2))
label_mapping_task2 = {0: 'MODY', 1: 'NDM', 2: 'Non-Monogenic'}

X_train_task2, X_test_task2, y_train_task2, y_test_task2 = train_test_split(
    X_task2, y_task2, test_size=0.3, random_state=42, stratify=y_task2

```

)

```
models_task2 = {
```

```
    "Logistic Regression": OneVsRestClassifier(LogisticRegression(max_iter=2000)),
```

```
    "Random Forest": RandomForestClassifier(random_state=42),
```

```
    "SVM": SVC(probability=True, random_state=42),
```

```
    "KNN": KNeighborsClassifier()
```

```
}
```

```
plt.figure(figsize=(10, 8))
```

```
for name, model in models_task2.items():
```

```
    pipeline = Pipeline(steps=[('preprocessor', preprocessor), ('classifier', model)])
```

```
    pipeline.fit(X_train_task2, y_train_task2)
```

```
    y_pred_task2 = pipeline.predict(X_test_task2)
```

```
    y_pred_prob_task2 = pipeline.predict_proba(X_test_task2) if hasattr(model, "predict_proba") else None
```

```
    print(f"\n{name} Performance (Task 2: MODY, NDM, Non-Monogenic):")
```

```
    print(f"Accuracy: {accuracy_score(y_test_task2, y_pred_task2):.4f}")
```

```
    print(f"Classification Report:\n{classification_report(y_test_task2, y_pred_task2, target_names=['MODY', 'NDM', 'Non-Monogenic'])}")
```

```
    if y_pred_prob_task2 is not None:
```

```
        roc_auc = roc_auc_score(y_test_task2, y_pred_prob_task2, multi_class="ovr")
```

```
        print(f"ROC-AUC: {roc_auc:.4f}")
```

```
        for i, label in label_mapping_task2.items():
```

```
            if i == 2:
```

```
                continue
```

```
            fpr, tpr, _ = roc_curve(y_test_task2 == i, y_pred_prob_task2[:, i])
```

```
            roc_auc_label = auc(fpr, tpr)
```

```
            plt.plot(fpr, tpr, label=f"{name} - {label} (AUC = {roc_auc_label:.2f})")
```

```
plt.plot([0, 1], [0, 1], 'k--', label="Random Guess")
```

```
plt.title("ROC Curve for Task 2: MODY and NDM Prediction")
```

```
plt.xlabel("False Positive Rate")
```

```
plt.ylabel("True Positive Rate")
```

```
plt.legend()
```

```

plt.show()
#Confusion matrices

from sklearn.metrics import confusion_matrix

import seaborn as sns

fig, ax = plt.subplots(2, 4, figsize=(20, 12))

for i, (name, model) in enumerate(models_task1.items()):

    pipeline = Pipeline(steps=[('preprocessor', preprocessor), ('classifier', model)])

    pipeline.fit(X_train_task1, y_train_task1)

    y_pred_task1 = pipeline.predict(X_test_task1)

    cm_task1 = confusion_matrix(y_test_task1, y_pred_task1)

    sns.heatmap(cm_task1, annot=True, fmt='d', cmap='Blues', ax=ax[0, i], cbar=False)

    ax[0, i].set_title(f'{name} Confusion Matrix (Task 1)')

    ax[0, i].set_xlabel('Predicted')

    ax[0, i].set_ylabel('True')

for i, (name, model) in enumerate(models_task2.items()):

    pipeline = Pipeline(steps=[('preprocessor', preprocessor), ('classifier', model)])

    pipeline.fit(X_train_task2, y_train_task2)

    y_pred_task2 = pipeline.predict(X_test_task2)

    cm_task2 = confusion_matrix(y_test_task2, y_pred_task2)

    sns.heatmap(cm_task2, annot=True, fmt='d', cmap='Blues', ax=ax[1, i], cbar=False)

    ax[1, i].set_title(f'{name} Confusion Matrix (Task 2)')

    ax[1, i].set_xlabel('Predicted')

    ax[1, i].set_ylabel('True')

plt.tight_layout()

plt.show()

# Visualizing accuracy of models

models = ['Logistic Regression', 'Random Forest', 'SVM', 'KNN']

task_1_accuracies = [0.9161, 0.9595, 0.9432, 0.9334]

task_2_accuracies = [0.9315, 0.9568, 0.9415, 0.9306]

fig, (ax1, ax2) = plt.subplots(nrows=1, ncols=2, figsize=(14, 6))

ax1.bar(models, task_1_accuracies, color='skyblue')

ax1.set_xlabel('Models')

ax1.set_ylabel('Accuracy (%)')

ax1.set_title('Task 1 (Monogenic Diabetes Prediction) Accuracy Comparison')

```

```

for i, v in enumerate(task_1_accuracies):
    ax1.text(i, v + 0.005, f'{v * 100:.1f}%', ha='center', va='bottom', fontsize=12)
ax1.set_ylim(0, 1.05)
ax2.bar(models, task_2_accuracies, color='lightgreen')
ax2.set_xlabel('Models')
ax2.set_ylabel('Accuracy (%)')
ax2.set_title('Task 2 (MODY, NDM prediction) Accuracy Comparison')
for i, v in enumerate(task_2_accuracies):
    ax2.text(i, v + 0.005, f'{v * 100:.1f}%', ha='center', va='bottom', fontsize=12)
ax2.set_ylim(0, 1.05)
plt.tight_layout(pad=4.0)
fig, ax = plt.subplots(figsize=(10, 6))
bar_width = 0.35
rects1 = ax.bar(np.arange(len(models)) - bar_width/2, task_1_accuracies, bar_width, label='Task 1 Accuracy', color='skyblue')
rects2 = ax.bar(np.arange(len(models)) + bar_width/2, task_2_accuracies, bar_width, label='Task 2 Accuracy', color='lightgreen')
ax.set_xlabel('Models')
ax.set_ylabel('Accuracy (%)')
ax.set_title('Model Accuracy Comparison for Task 1 and Task 2')
ax.set_xticks(np.arange(len(models)))
ax.set_xticklabels(models)
ax.legend(loc='upper left', bbox_to_anchor=(1, 1))
def add_labels(rects):
    for rect in rects:
        height = rect.get_height()
        ax.annotate(f'{height * 100:.1f}%',
                    xy=(rect.get_x() + rect.get_width() / 2, height),
                    xytext=(0, 3),
                    textcoords="offset points",
                    ha='center', va='bottom', fontsize=12)

add_labels(rects1)
add_labels(rects2)
plt.tight_layout(pad=4.0)
plt.show()
# K-fold cross-validation for the best performing model (Random Forest)

```

```

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
rf_model = RandomForestClassifier(random_state=42)
pipeline = Pipeline(steps=[('preprocessor', preprocessor), ('classifier', rf_model)])
task1_fold_scores = []
task2_fold_scores = []

plt.figure(figsize=(15, 6))
plt.subplot(1, 2, 1)
for fold, (train_idx, val_idx) in enumerate(cv.split(X_task1, y_task1), start=1):
    X_train_fold = X_task1.iloc[train_idx]
    y_train_fold = y_task1.iloc[train_idx]
    X_val_fold = X_task1.iloc[val_idx]
    y_val_fold = y_task1.iloc[val_idx]
    pipeline.fit(X_train_fold, y_train_fold)
    y_pred_proba = pipeline.predict_proba(X_val_fold)[:, 1]
    fold_score = roc_auc_score(y_val_fold, y_pred_proba)
    task1_fold_scores.append(fold_score)

plt.plot(fold, fold_score, 'bo-', markersize=8, label='Fold scores' if fold == 1 else '')
mean_score_task1 = np.mean(task1_fold_scores)
std_score_task1 = np.std(task1_fold_scores)
plt.plot(range(1, 6), task1_fold_scores, 'b-', label='ROC-AUC scores')
plt.axhline(y=mean_score_task1, color='r', linestyle='--', label=f'Mean ROC-AUC = {mean_score_task1:.4f}')
plt.fill_between(range(1, 6),
                 mean_score_task1 - std_score_task1,
                 mean_score_task1 + std_score_task1,
                 color='b', alpha=0.2, label=f'Std: ±{std_score_task1:.4f}')
y_min_task1 = min(task1_fold_scores) - 0.001
y_max_task1 = max(task1_fold_scores) + 0.001
plt.ylim(y_min_task1, y_max_task1)
plt.title("Task 1: Monogenic Diabetes Prediction\nK-Fold Cross-Validation")
plt.xlabel('Fold')
plt.ylabel('ROC-AUC Score')
plt.legend()
plt.grid(True)

```

```

plt.subplot(1, 2, 2)
for fold, (train_idx, val_idx) in enumerate(cv.split(X_task2, y_task2), start=1):
    X_train_fold = X_task2.iloc[train_idx]
    y_train_fold = y_task2.iloc[train_idx]
    X_val_fold = X_task2.iloc[val_idx]
    y_val_fold = y_task2.iloc[val_idx]
    pipeline.fit(X_train_fold, y_train_fold)
    y_pred_proba = pipeline.predict_proba(X_val_fold)
    fold_score = roc_auc_score(y_val_fold, y_pred_proba, multi_class='ovr', average='macro')
    task2_fold_scores.append(fold_score)

plt.plot(fold, fold_score, 'go-', markersize=8, label='Fold scores' if fold == 1 else '')
mean_score_task2 = np.mean(task2_fold_scores)
std_score_task2 = np.std(task2_fold_scores)
plt.plot(range(1, 6), task2_fold_scores, 'g-', label='ROC-AUC scores')
plt.axhline(y=mean_score_task2, color='orange', linestyle='--', label=f'Mean ROC-AUC = {mean_score_task2:.4f}')
plt.fill_between(range(1, 6),
                 mean_score_task2 - std_score_task2,
                 mean_score_task2 + std_score_task2,
                 color='g', alpha=0.2, label=f'Std: ± {std_score_task2:.4f}')
y_min_task2 = min(task2_fold_scores) - 0.001
y_max_task2 = max(task2_fold_scores) + 0.001

plt.ylim(y_min_task2, y_max_task2)
plt.title("Task 2: MODY and NDM Classification\nK-Fold Cross-Validation")
plt.xlabel('Fold')
plt.ylabel('ROC-AUC Score')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

```

References

1. Hattersley, A.T. and K.A. Patel, *Precision diabetes: learning from monogenic diabetes*. Diabetologia, 2017. **60**(5): p. 769-777.
2. Batra, A., *Comprehensive Diabetes Dataset with Genetic, Environmental, and Lifestyle Factors*. 2024.
3. Shepherd, M., et al., *Systematic Population Screening, Using Biomarkers and Genetic Testing, Identifies 2.5% of the U.K. Pediatric Diabetes Population With Monogenic Diabetes*. Diabetes Care, 2016. **39**(11): p. 1879-1888.
4. Wędrychowicz, A., et al., *Phenotype heterogeneity in glucokinase-maturity-onset diabetes of the young (GCK-MODY) patients*. Journal of clinical research in pediatric endocrinology, 2017. **9**(3): p. 246.
5. Shields, B.M., et al., *The development and validation of a clinical prediction model to determine the probability of MODY in patients with young-onset diabetes*. Diabetologia, 2012. **55**(5): p. 1265-1272.
6. Chakera, A.J., et al., *Recognition and Management of Individuals With Hyperglycemia Because of a Heterozygous Glucokinase Mutation*. Diabetes Care, 2015. **38**(7): p. 1383-1392.